Discovery AI

Signature Discovery

Professor Iain Styles (i.styles@qub.ac.uk)

School of EEECS, QUB

November 9 2015

Outline

1. Introduction
   - About the Module
   - Signature Discovery
2. Prototype Methods
   - Multivariate data
   - Supervised Learning of Prototypes with LDA
   - k-mean clustering
3. Decomposition-based Approaches
   - Principal Component Analysis
   - Non-Negative Matrix Factorisation

What this Module is about

The course will cover a series of topics e.g.

1. Signature Discovery
   - Introduction: Data signatures are distinctive, domain-specific patterns that characterize objects, events, or processes.
   - Classical Methods: Techniques like clustering, feature selection, and discriminative modelling to identify salient patterns.
   - Modern Methods: Detection and representation learning approaches that characterize objects without relying solely on class labels.
   - Case Studies: Viral signature discovery in biomedical data for diagnosis and monitoring.
2. Causality Discovery
   - Introduction: Distinguishing causation from correlation is essential for reliable inference and decision-making.
   - Classical Methods: SCMs, Granger causality, constraint-based (e.g., PC) and score-based (e.g., GES) approaches.
   - Modern Methods: Deep learning models like CausalFormer and world models for learning causal structure, interventions, and counterfactuals.
   - Case Studies: Drug target discovery in biomedical research.
3. Hypothesis Discovery
   - Introduction: Hypothesis generation is central to exploration, experimentation, and theory building.
   - Classical Methods: Abductive reasoning to infer the most plausible explanations under uncertainty.
   - Modern Methods: LLMs and world models to suggest relationships, mechanisms, and counterfactuals.
   - Case Studies: Investigating biological mechanisms of ageing using AI-driven hypothesis discovery.

Learning Outcomes

Successful students will be able to:

1. Demonstrate an understanding of key concepts and methodologies related to signature discovery, causality discovery, and hypothesis generation.
2. Apply techniques from signature discovery to extract unique identifiers from diverse datasets.
3. Analyse causal relationships within data to distinguish correlation from causation in AI applications.
4. Generate data-driven hypotheses and evaluate their scientific validity using AI-based tools.

Lectures and Practicals

All sessions will all take place from 10am-4pm in the Allstate Software Studio on the 2nd Floor of the Computer Science Building.

- Tuesday 10th February
- Friday 20th February
- Tuesday 24th February
- Tuesday 3rd March
- Tuesday 10th March
- Friday 20th March

Assessment

- Students must demonstrate active engagement in a minimum of 80% of the timetabled activities to pass the course, which is assessed as pass/fail only. Must be passed to pass the course.
- Individual written report, graded as pass/fail. Must be passed to pass the course.

Signature Discovery

The first part of the course is on **Signature Discovery**

Signature Discovery

The first part of the course is on **Signature Discovery**

What it is about

- What "signatures" are? (warning – no universal definition)
- Why signatures are important
- How we can approach the signature discovery process

Signature Discovery

The first part of the course is on **Signature Discovery**

What it is about

- What "signatures" are? (warning – no universal definition)
- Why signatures are important
- How we can approach the signature discovery process

What it is not about

- Detailed descriptions of methods
- Prescriptive recipes on how to identify signatures
- How to use these ideas in your PhD projects

Signatures?

- What do we mean when we talk about "signatures"?

Signatures?

- What do we mean when we talk about "signatures"?
- A factor or set of factors that define or identify some (biological) phenomenon

Signatures vs Biomarkers

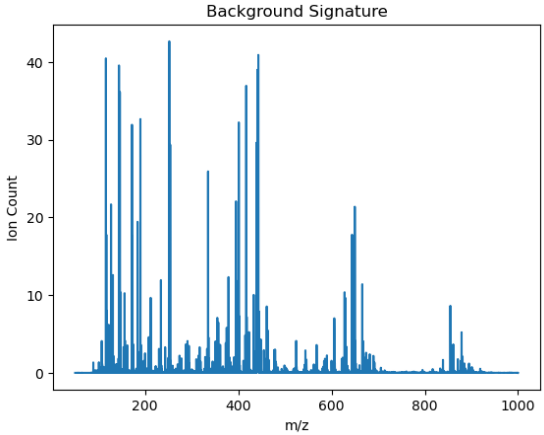
Biomarkers

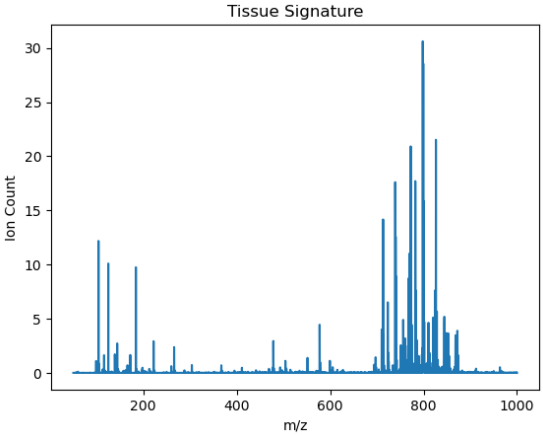
- Biomarkers are objective and measurable
- Usually corresponding to single factors (a gene, molecule etc)
- Predictive or diagnostic power
- Established causal link, well characterised values under normal conditions

Signatures

- Signatures can represent combinations of factors
- Not always directly measurable
- Not always informed by causal links
- Not always selected for predictive/diagnostic ability
- BUT – often informative about underlying phenomena – **discovery**

Examples of Signatures





How will we approach this?

- "A factor or set of factors that define or identify some (biological) phenomenon"
- Factors $\implies$ variables
- Signature: pattern of/relationship between variables
- Signature discovery: process for identifying those patterns

Categorization of approaches

We will look at four approaches divided into two classes

Prototype-based

- Learn "typical" examples of a phenomenon
- From data that is labelled (supervised)
- From data that is not labelled (unsupervised)

Decomposition-based

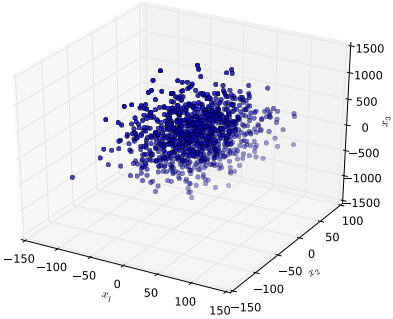
- Learn factors that be used to represent the data
- Principal Component Analysis
- Non-negative matrix factorisation

We will then draw these together to examine how to interpret the signatures we find.

Prototype-based Methods

- Consider a dataset in which a number of phenomena are know to be represented
  - Different cell/tissue types
  - Healthy/unhealthy samples
  - Samples acquired under different conditions
- Identifying a "typical" example of each phenomenon present constitutes a **signature**
- Signatures may allow us to learn something important
  - Distinguishing features of a phenomenon
  - Differences between phenomena
- How we do find these signatures?

A Brief Interlude: Multivariate Data

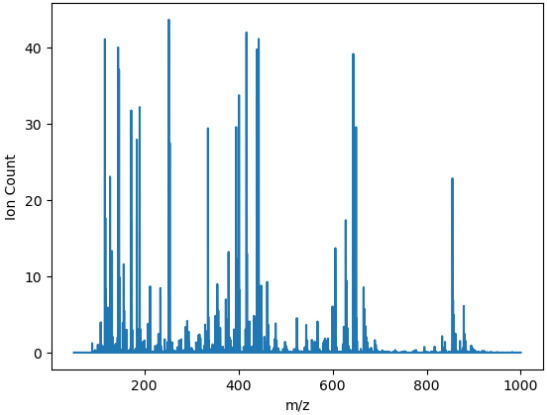


- Each sample is represented by a collection of numbers
- Each number represents a "feature" of the data

Examples of Multivariate Data

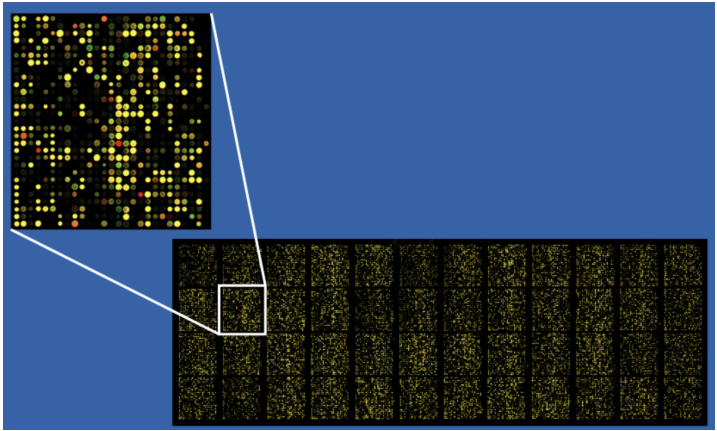
What are the variables?

Examples of Multivariate Data



What are the variables?

Examples of Multivariate Data

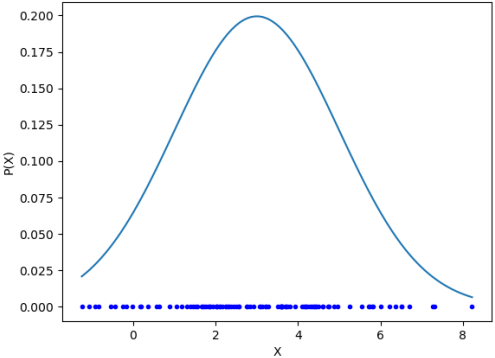


What are the variables?

Variance

- A single random variable X is often characterised by its expected value (mean) $\mu = \mathrm{E}(x)$, and its variance.
- Variance characterises the range of possible value that X can take and is defined as

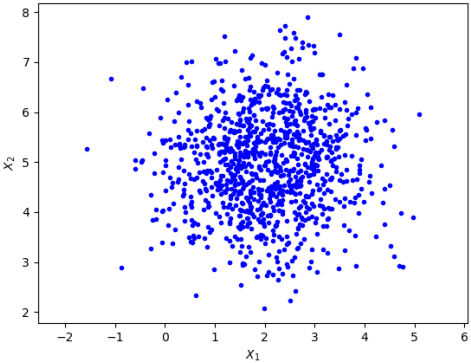
$$\mathrm{var}(X) = \sigma^2 = \sum_{i=1}^{N} (X_i - \mu)^2$$



Covariance

- A vector of random variables $\mathbf{X} = (X_1, \ldots, X_N)$ is characterised by its expected value (mean) $\boldsymbol{\mu} = \mathrm{E}(\mathbf{X}) = \mu_1 \ldots \mu_N$, and its *covariance* $\boldsymbol{\Sigma}$.

$$\Sigma_{ij} = \frac{1}{N-1} \sum_{n=1}^{N} \left(X_i^{(n)} - \mu_i \right) \left(X_j^{(n)} - \mu_j \right)$$

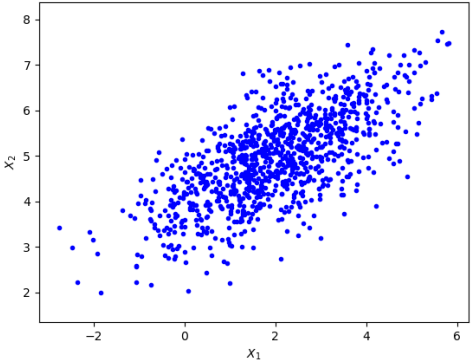


$$\boldsymbol{\Sigma} = \begin{pmatrix} 1.0 & 0.0 \\ 0.0 & 1.0 \end{pmatrix}$$

Covariance

- A vector of random variables $\mathbf{X} = (X_1, \ldots, X_N)$ is characterised by its expected value (mean) $\boldsymbol{\mu} = \mathrm{E}(\mathbf{X}) = \mu_1 \ldots \mu_N$, and its *covariance* $\boldsymbol{\Sigma}$.

$$\Sigma_{ij} = \frac{1}{N-1} \sum_{n=1}^{N} \left(X_i^{(n)} - \mu_i \right) \left(X_j^{(n)} - \mu_j \right)$$

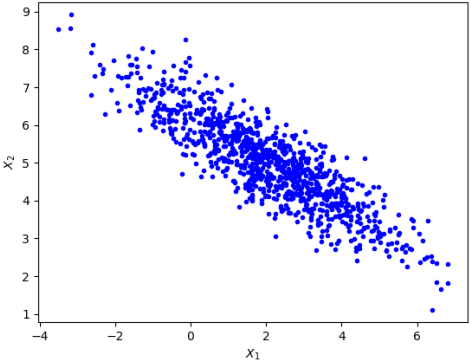


$$\boldsymbol{\Sigma} = \begin{pmatrix} 2.0 & 1.0 \\ 1.0 & 1.0 \end{pmatrix}$$

Covariance

- A vector of random variables $\mathbf{X} = (X_1, \ldots, X_N)$ is characterised by its expected value (mean) $\boldsymbol{\mu} = \mathrm{E}(\mathbf{X}) = \mu_1 \ldots \mu_N$, and its *covariance* $\boldsymbol{\Sigma}$.

$$\Sigma_{ij} = \frac{1}{N-1} \sum_{n=1}^{N} \left(X_i^{(n)} - \mu_i \right) \left(X_j^{(n)} - \mu_j \right)$$



$$\boldsymbol{\Sigma} = \begin{pmatrix} 3.0 & -2.0 \\ -2.0 & 1.0 \end{pmatrix}$$

Supervised Approach to Prototype Learning

- Consider a dataset of N samples, each of which has M features.
- The dataset contains examples of different phenomena
- Some or all of the samples are labelled with the phenomenon they represent
- There are labelled samples from all of the known phenomena

Supervised Approach to Prototype Learning

- Consider a dataset of N samples, each of which has M features.
- The dataset contains examples of different phenomena
- Some or all of the samples are labelled with the phenomenon they represent
- There are labelled samples from all of the known phenomena
- For each of the known categories/phenomena we will learn a **prototype**
- A "data point" that captures the core characteristics of the phenomenon

Supervised Approach to Prototype Learning

- Consider a dataset of N samples, each of which has M features.
- The dataset contains examples of different phenomena
- Some or all of the samples are labelled with the phenomenon they represent
- There are labelled samples from all of the known phenomena
- For each of the known categories/phenomena we will learn a **prototype**
- A "data point" that captures the core characteristics of the phenomenon
- In other words, as **signature**

Prototype learning with Classifiers

- We have labelled examples of each phenomenon
- We can therefore learn prototypes by training a **classifier**
- Use the labelled samples to construct:
  - A classification decision boundary to label unlabelled samples with the correct phenomenon
  - A "typical" example of the category
- Which classifier?

Prototype learning with Classifiers

- We have labelled examples of each phenomenon
- We can therefore learn prototypes by training a **classifier**
- Use the labelled samples to construct:
  - A classification decision boundary to label unlabelled samples with the correct phenomenon
  - A "typical" example of the category
- Which classifier?

- Neural network (many types)
- Support Vector Machine
- Random Forest
- Logistic Regression
- Decision Tree
- Naive Bayes
- k-nearest neighbour

Prototype learning with Classifiers

- We have labelled examples of each phenomenon
- We can therefore learn prototypes by training a **classifier**
- Use the labelled samples to construct:
  - A classification decision boundary to label unlabelled samples with the correct phenomenon
  - A "typical" example of the category
- Which classifier?

- Neural network (many types)
- Support Vector Machine
- Random Forest
- Logistic Regression
- Decision Tree
- Naive Bayes
- k-nearest neighbour
- **(Linear) Discriminant Analysis**

Why choose LDA?

For

- Clear and transparent underlying model
- Mathematically straightforward
- Prototypes emerge naturally
- Explicitly generative
- Easy to implement
- (Lecturer's personal preference)

Why choose LDA?

For

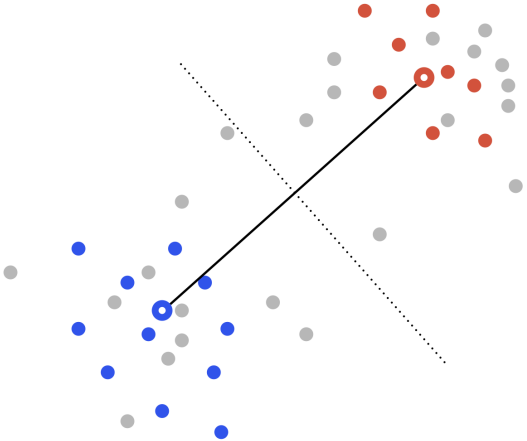
- Clear and transparent underlying model
- Mathematically straightforward
- Prototypes emerge naturally
- Explicitly generative
- Easy to implement
- (Lecturer's personal preference)

Against

- Restrictive assumptions
- Requires explicit mathematical understanding
- Not sexy/trendy
  - (But you can apply the ideas to other more "modern" approaches)

The Basic Idea

- Given the labelled data points:
- Construct a *classification rule* as some combination of the input variables that separates the predefined classes as much as possible: **Discrimination**.
- Use the classifier to *predict* the class of new unlabelled data points: **Classification**.



How LDA does this

- Construct a *statistical model* of each class
- Each class defined by its *mean* and *covariance*
- Compute the line of equal probability
- Classify points according to which side of the line they are

How LDA does this

- Construct a *statistical model* of each class
- Each class defined by its *mean* and *covariance*
- Compute the line of equal probability
- Classify points according to which side of the line they are
- **The statistics define the signature of the class**

Quick Review of Conditional Probability

We will treat this as a statistical problem, making liberal use of **Bayes' theorem**:

$$P(a|b) = P(b|a)P(a)/P(b)$$

- $P(a|b)$ is the probability of a being true, given that b is already true.
- $P(b|a)$ is the probability of b being true, given that a is already true.
- $P(a)$ and $P(b)$ are the probabilities of a and b being true, irrespective of prior knowledge.

Conditional Probability

Example: chemical "assay" that determines the presence of a particular chemical in a sample. We know the following::

- **Sensitivity** (true positive rate) is 90%: if a sample contains the chemical, there is a 90% chance of the test being positive.
- **Specificity** (true negative rate) is 80%: if a sample does not contain the chemical, there is an 80% chance of the test being negative.
- It is **expected (prior knowledge)** that 10% of samples contain the chemical.

Conditional Probability

How does this affect test interpretation? Letting T be the test and S the sample we have:

- Sensitivity of 90%

$$\mapsto P(+T|+S) = 0.9 \text{ and } P(-T|+S) = 0.1$$

- Specificity of 80%

$$\mapsto P(-T|-S) = 0.8 \text{ and } P(+T|-S) = 0.2$$

- Prior knowledge

$$\mapsto P(+S) = 0.1 \text{ and hence } P(-S) = 0.9$$

[sample must always give a positive or negative test result; probabilities must sum to one]

Asking Questions with Conditional Probabilities

What is the probability that a sample contains the chemical given a positive test result?

- From Bayes' theorem:

$$P(+S|+T) = P(+T|+S)P(+S)/P(+T)$$

- Need probability of positive test, $P(+T)$.
- Two ways to get T: a false positive from negative sample is negative; true positive from positive sample:

$$\begin{aligned} P(+T) &= P(+T|+S)P(+S) + P(+T|-S)P(-S) \\ &= (0.9 \times 0.1) + (0.2 \times 0.9) = 0.27 \end{aligned}$$

- Similarly for a negative test:

$$P(-T) = 0.73.$$

Asking Questions with Conditional Probabilities

- Then we have

$$P(+S|+T) = 0.9 \times 0.1/0.27 = 0.333$$

and similarly

$$\begin{aligned} P(+S|-T) &= 0.1 \times 0.1/0.73 = 0.014 \\ P(-S|+T) &= 0.2 \times 0.9/0.27 = 0.667 \\ P(-S|-T) &= 0.9 \times 0.9/0.73 = 0.986 \end{aligned}$$

Implications

- If a test is negative, 98.6% prob that the sample does not contain the chemical.
- If the test is positive, it is **twice as likely** that the sample will be negative than positive.
- Surprising given the high sensitivity of the test
- Must take into account the relative sizes of the two populations
- Only 10% samples likely to be positive
- Much more likely that positives are false positives (90% of samples ae negative)

Linear Discriminant Analysis

- Bayesian statistics will allow us to classify data in a principled way
- Allows us to calculate probabilities of points belonging to groups
- Simplest base: linear binary classifier
- Linear combination of variables–a straight line–that best discriminates between two pre-defined groups of points
- Uses the probability distributions of the classes to find the line of equal probability dividing two classes

Formulating LDA

- Consider a data point $\mathbf{x}$ which we want to assign to one of two predefined classes Π_i $(i = 1, 2)$
- We write that $P(\mathbf{x} \in \Pi_i) = \pi_i$
- We also define the conditional probability $P(\mathbf{x}|\Pi_i) = f_i(\mathbf{x})$, the distribution of points given class Π_i
- Then the probability that a given point $\mathbf{x}$ belongs to class Π_i is, by Bayes rule:

$$\begin{aligned} P(\Pi_i|\mathbf{x}) &= \frac{P(\mathbf{x}|\Pi_i)P(\Pi_i)}{P(\mathbf{x})} \\ &= \frac{f_i(\mathbf{x})\pi_i}{f_1(\mathbf{x})\pi_1 + f_2(\mathbf{x})\pi_2} \end{aligned}$$

Formulating LDA

- Give the distributions of the groups $f_i(\mathbf{x})$, and the probabilities $\pi_x(\mathbf{x})$ of a point being in the groups.
- New points are assigned to the group with the highest probability:

$$\frac{P(\Pi_1|\mathbf{x})}{P(\Pi_2|\mathbf{x})} > 1 \mapsto \mathbf{x} \in \Pi_1 \quad \text{else}\, \mathbf{x} \in \Pi_2$$

- In terms of f and π, $\mathbf{x}$ belongs to Π_1 if

$$\frac{f_1(\mathbf{x})}{f_2(\mathbf{x})} > \frac{\pi_1}{\pi_2}$$

- otherwise to Π_2.
- Equal ratios, randomly assign to either class.

Defining the Classes

- We need to define the class distributions $f_i(\mathbf{x})$.
- For a simple two-class binary classifier we assume:
  - Each data point belongs to exactly one of exactly two distinct and identifiable groups, Π_1 and Π_2
  - The two groups are normally distributed with different means $\bar{\mathbf{x}}_1$ and $\bar{\mathbf{x}}_2$ but identical covariances $\mathbf{\Sigma}$.
- Thus, for $n = \{1, 2\}$ the groups distributions are $P(\mathbf{x}|\Pi_i) = f_i(\mathbf{x})$:

$$f_n(\mathbf{x}) = \frac{1}{(2\pi)^{r/2}\,|\mathbf{\Sigma}|^{1/2}} \exp\left[-\frac{1}{2}(\mathbf{x} - \bar{\mathbf{x}}_n)^{\mathrm{T}}\mathbf{\Sigma}^{-1}(\mathbf{x} - \bar{\mathbf{x}}_n)\right]$$

The Separation Rule

- We know the ratio $\frac{f_1(\mathbf{x})\pi_2}{f_2(\mathbf{x})\pi_1} = 1$ separates the groups.
- Taking logs: $\log_e \frac{f_1(\mathbf{x})\pi_2}{f_2(\mathbf{x})\pi_1} = \log_e \frac{f_1(\mathbf{x})}{f_2(\mathbf{x})} - \log_e \frac{\pi_1}{\pi_2}$ with:

$$\log_e \frac{f_1(\mathbf{x})}{f_2(\mathbf{x})} = (\bar{\mathbf{x}}_1 - \bar{\mathbf{x}}_2)^{\mathrm{T}} \boldsymbol{\Sigma}^{-1}\mathbf{x} - \frac{1}{2}(\bar{\mathbf{x}}_1 - \bar{\mathbf{x}}_2)^{\mathrm{T}} \boldsymbol{\Sigma}^{-1}(\bar{\mathbf{x}}_1 + \bar{\mathbf{x}}_2)$$

- The second term on the RHS of this equation is "constant" (no $\mathbf{x}$) and simplifies as

$$(\bar{\mathbf{x}}_1 - \bar{\mathbf{x}}_2)^{\mathrm{T}} \boldsymbol{\Sigma}^{-1}(\bar{\mathbf{x}}_1 + \bar{\mathbf{x}}_2) = \bar{\mathbf{x}}_1^{\mathrm{T}}\boldsymbol{\Sigma}^{-1}\bar{\mathbf{x}}_1 - \bar{\mathbf{x}}_2^{\mathrm{T}}\boldsymbol{\Sigma}^{-1}\bar{\mathbf{x}}_2$$

The Separation Rule

- Writing $\mathbf{m} = \mathbf{\Sigma}^{-1}(\bar{\mathbf{x}}_1 - \bar{\mathbf{x}}_2)$
- And $c = -\frac{1}{2}\left(\bar{\mathbf{x}}_1^{\mathrm{T}}\mathbf{\Sigma}^{-1}\bar{\mathbf{x}}_1 - \bar{\mathbf{x}}_2^{\mathrm{T}}\mathbf{\Sigma}^{-1}\bar{\mathbf{x}}_2\right) + \log_e \frac{\pi_1}{\pi_2}$
- We have

$$L(\mathbf{x}) = \log_e \frac{f_1(\mathbf{x})\pi_1}{f_2(\mathbf{x})\pi_2} = \mathbf{m}^{\mathrm{T}}\mathbf{x} + c,$$

- Which yields the following decision rule:

$$\text{if } L(\mathbf{x}) > 0 \text{ assign } \mathbf{x} \text{ to } \Pi_1 \text{ else } \Pi_2$$

- This is *Gaussian LDA*
- $\mathbf{m}^{\mathrm{T}}\mathbf{x}$ is *Fisher's linear discriminant function*.

Assumptions of LDA

1. There are two classes
2. There are sufficient labelled examples to estimate the class statistics
3. Classes are cleanly separated by a linear boundary
4. Both classes are normally distributed
5. Both classes having the same covariance

Assumptions of LDA

1. There are two classes
2. There are sufficient labelled examples to estimate the class statistics
3. Classes are cleanly separated by a linear boundary
4. Both classes are normally distributed
5. Both classes having the same covariance

There are extensions of LDA to address assumptions 1, 3 and 5
In practice, some relaxation of conditions 4, 5 is possible without too much impact.

Quadratic Discriminant Analysis

- Different class covariances require more complex analysis
- If the covariances of the two classes are $\mathbf{\Sigma}_1$ and $\mathbf{\Sigma}_2$ the discriminant becomes:

$$Q(\mathbf{x}) = \mathbf{x}^{\mathrm{T}}\mathbf{A}\mathbf{x} + \mathbf{b}^{\mathrm{T}}\mathbf{x} + c$$

a *Quadratic discriminant* with

$$\begin{aligned}
\mathbf{A} &= -\frac{1}{2}\left(\mathbf{\Sigma}_1^{-1} - \mathbf{\Sigma}_2^{-1}\right) \\
\mathbf{b} &= \mathbf{\Sigma}_1^{-1}\bar{\mathbf{x}}_1 - \mathbf{\Sigma}_2^{-1}\bar{\mathbf{x}}_2 \\
c &= -\frac{1}{2}\left(\log_e \frac{|\mathbf{\Sigma}_1|}{|\mathbf{\Sigma}_2|} + \bar{\mathbf{x}}_1^{\mathrm{T}}\mathbf{\Sigma}_1^{-1}\bar{\mathbf{x}}_1 - \bar{\mathbf{x}}_2^{\mathrm{T}}\mathbf{\Sigma}_2^{-1}\bar{\mathbf{x}}_2\right) - \log_e \frac{\pi_1}{\pi_2}
\end{aligned}$$

The classification rule is:

$$\text{if } Q(\mathbf{x}) > 0 \text{ assign } \mathbf{x} \text{ to } \Pi_1$$

else assign $\mathbf{x}$ to Π_2.

Multiclass LDA

- Generalisation multiclass data require significantly more algebra. . . .
- Compute the pairwise relative probabilities as before and form the discriminant

$$L_{ij}(\mathbf{x}) = \log_e \left(\frac{P(\Pi_i|\mathbf{x})}{P(\Pi_j|\mathbf{x})} \right) = \log_e \left(\frac{f_i(\mathbf{x})\pi_i}{f_j(\mathbf{x})\pi_j} \right)$$

- $\mathbf{x}$ is assigned to Π_i if $L_{IJ} > 0$ for all $j \neq i$.
- The discriminant function between classes i and j is then

$$L_{ij}(\mathbf{x}) = \mathbf{m}_{ij}^{\mathrm{T}}\mathbf{x} + c_{ij}$$

with

$$\begin{aligned} \mathbf{m}_{ij} &= (\bar{\mathbf{x}}_i - \bar{\mathbf{x}}_j)^{\mathrm{T}} \boldsymbol{\Sigma}^{-1} \text{ and} \\ c_{ij} &= -\frac{1}{2} \left(\bar{\mathbf{x}}_i^{\mathrm{T}} \boldsymbol{\Sigma}^{-1} \bar{\mathbf{x}}_i - \bar{\mathbf{x}}_j^{\mathrm{T}} \boldsymbol{\Sigma}^{-1} \bar{\mathbf{x}}_j \right) + \log_e \frac{\pi_i}{\pi_j}. \end{aligned}$$

Considerations

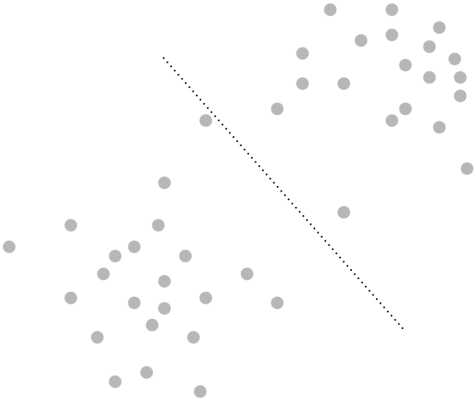
- Scales matter when calculating statistical patterns
- Height in metres vs mm; mass in tonnes or ng
- It is common to normalise variables to have the same scale

Considerations

- Scales matter when calculating statistical patterns
- Height in metres vs mm; mass in tonnes or ng
- It is common to normalise variables to have the same scale
  - Same min/max if bounded
  - Same variance if unbound
  - Other approaches exist

Prototype Learning with Clustering

- What do we do if we do not have any labelled examples of our phenomena?
- We cannot use class statistics to derive the boundaries
- We have to find the structure ourselves
- Clustering



Clustering

- We have a multivariate dataset containing samples representing multiple phenomena
- We have no labelled examples
- In classification, we group datapoints based on their similarity to the labelled samples
- In clustering we have to find those groups without the labels
- We will aim to find:
  - A set of boundaries that separate distinct phenomena
  - A "typical" example of the category
- Many clustering methods - which should be use?

Clustering

- We have a multivariate dataset containing samples representing multiple phenomena
- We have no labelled examples
- In classification, we group datapoints based on their similarity to the labelled samples
- In clustering we have to find those groups without the labels
- We will aim to find:
  - A set of boundaries that separate distinct phenomena
  - A "typical" example of the category
- Many clustering methods - which should be use?
- Hierarchical clustering
- DBScan
- ToMATo
- Gaussian mixtures
- Affinity Propagation
- Spectral Clustering

Clustering

- We have a multivariate dataset containing samples representing multiple phenomena
- We have no labelled examples
- In classification, we group datapoints based on their similarity to the labelled samples
- In clustering we have to find those groups without the labels
- We will aim to find:
  - A set of boundaries that separate distinct phenomena
  - A "typical" example of the category
- Many clustering methods - which should be use?

- Hierarchical clustering
- DBScan
- ToMATo
- Gaussian mixtures
- Affinity Propagation
- Spectral Clustering
- ***k*-means**

Why choose k-means?

For

- Clear and transparent underlying model
- Computationally straightforward
- Prototypes emerge naturally
- Can be made generative
- Scales well
- (Lecturer's personal preference)

Why choose k-means?

For

- Clear and transparent underlying model
- Computationally straightforward
- Prototypes emerge naturally
- Can be made generative
- Scales well
- (Lecturer's personal preference)

Against

- Restrictive assumptions
- Practical implementation relies on heuristic
- Not sexy/trendy
  - (But you can apply the ideas to other more "modern" approaches)

The k-means algorithm

- In k-means clustering we aim to partition set of n observations into $k \leq n$ sets $\mathbf{S} = \{S1, S2, ..., S_k\}$
- This is done to minimize the within-cluster sum of squares (the variance)
- In other words, we try to find the k "tightest" clusters
- Formally this requires us to find:

$$\arg\min_{\mathbf{S}} \sum_{i=1}^{k} \sum_{\mathbf{x} \in \mathbf{S_i}} \|\mathbf{x} - \boldsymbol{\mu}_i\|^2$$

The k-means algorithm

- In k-means clustering we aim to partition set of n observations into $k \leq n$ sets $\mathbf{S} = \{S1, S2, ..., S_k\}$
- This is done to minimize the within-cluster sum of squares (the variance)
- In other words, we try to find the k "tightest" clusters
- Formally this requires us to find:

$$\arg\min_{\mathbf{S}} \sum_{i=1}^{k} \sum_{\mathbf{x} \in \mathbf{S_i}} \|\mathbf{x} - \boldsymbol{\mu}_i\|^2$$

- This is a surprisingly hard problem to solve in general
  - Combinatorial optimisation does not scale well
- In practice we solve using a heuristic

The k-means Algorithm

Informally...

- Number of clusters k is specified by the "user".
  1) First, randomly assign every data point to one of the clusters.
  2) Compute the mean position (in the data space) of each cluster
  3) For each data point, calculate distance to mean of each cluster
  4) Re-assign points to their nearest cluster.
  5) Return to step 2 and repeat until the clusters are not changing.

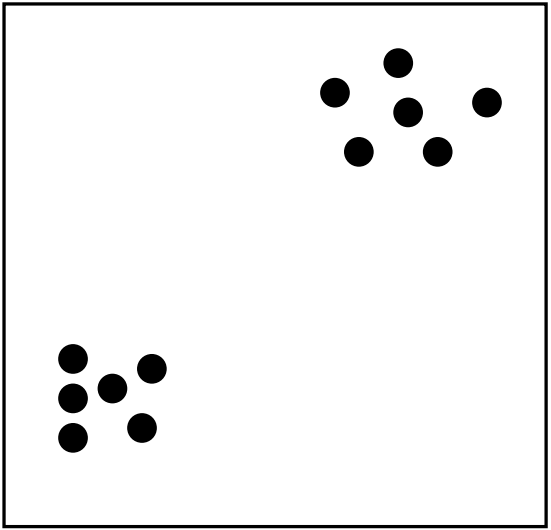
Lloyd's Algorithm

```
Data: Data points Data[N]. Number of
      clusters K
Result: μ, r(μ), parameters that
        minimise the least squares
        error, the error
% Randomly assign each point to a
  cluster
for i ← 1 to N do
    ClusterIndex[i] ← randint(1, K)
end
% Calculate the mean of each
  cluster
for i ← 1 to K do
    ClusterMean[i] ←
      mean(Data[ClusterIndex[i]])
end
% Iterate until clusters are
  stable
Converged ← False;
while Converged == False do
    % Compute distance from every
      point to each cluster mean
    for i ← 1 to K do
        for j ← 1 to N do
            dist[i][j] ←
              distance(ClusterMean[i], Data[j])
        end
    end
    % Find the nearest mean for
      each data point
    for j ← 1 to N do
        NewClusterIndex[j] ←
          nearest(Data[j], ClusterMean[j])
        end
    end
    % Generate new clusters
    for i ← 1 to K do
        ClusterMean[i] ←
          mean(Data[NewClusterIndex ==
          i])
    end
    % Check to see if cluster
      membership has changed
    Converged ← True
    if NewClusterIndex != ClusterIndex
      then
        Converged ← False
    end
    % Replace old clusters with
      new ones
    ClusterIndex ← NewClusterIndex
    ClusterMean ← NewClusterMean
end
```

Algorithm 1: The k-Means Algorithm.

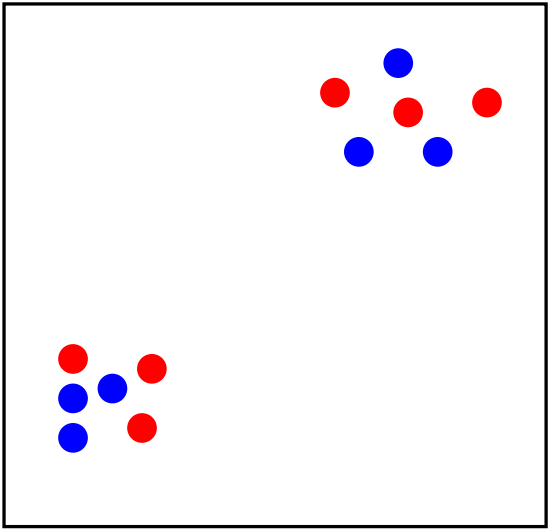
Lloyd's Algorithm Visualised

1) First, randomly assign every data point to one of the clusters.
2) Compute the mean position (in the data space) of each cluster
3) For each data point, calculate distance to mean of each cluster
4) Re-assign points to their nearest cluster.
5) Return to step 2 and repeat until the clusters are not changing.



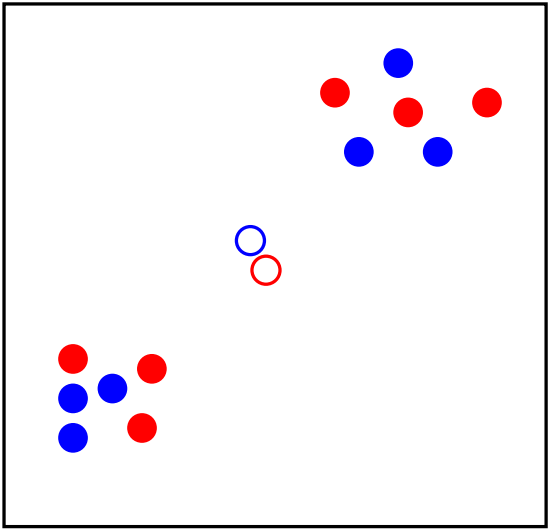
Lloyd's Algorithm Visualised

1) **First, randomly assign every data point to one of the clusters.**
2) Compute the mean position (in the data space) of each cluster
3) For each data point, calculate distance to mean of each cluster
4) Re-assign points to their nearest cluster.
5) Return to step 2 and repeat until the clusters are not changing.



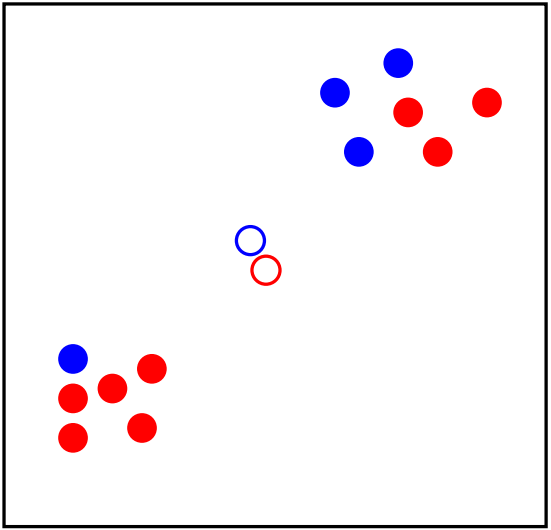
Lloyd's Algorithm Visualised

1) First, randomly assign every data point to one of the clusters.
2) **Compute the mean position (in the data space) of each cluster**
3) For each data point, calculate distance to mean of each cluster
4) Re-assign points to their nearest cluster.
5) Return to step 2 and repeat until the clusters are not changing.



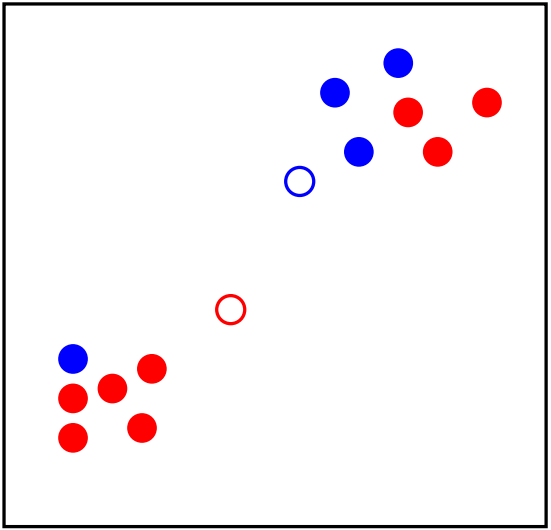
Lloyd's Algorithm Visualised

1) First, randomly assign every data point to one of the clusters.
2) Compute the mean position (in the data space) of each cluster
3) **For each data point, calculate distance to mean of each cluster**
4) **Re-assign points to their nearest cluster.**
5) Return to step 2 and repeat until the clusters are not changing.



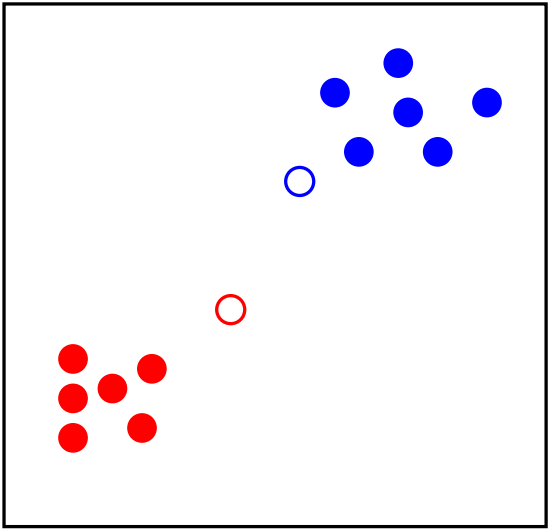
Lloyd's Algorithm Visualised

1) First, randomly assign every data point to one of the clusters.
2) **Compute the mean position (in the data space) of each cluster**
3) For each data point, calculate distance to mean of each cluster
4) Re-assign points to their nearest cluster.
5) **Return to step 2 and repeat until the clusters are not changing.**



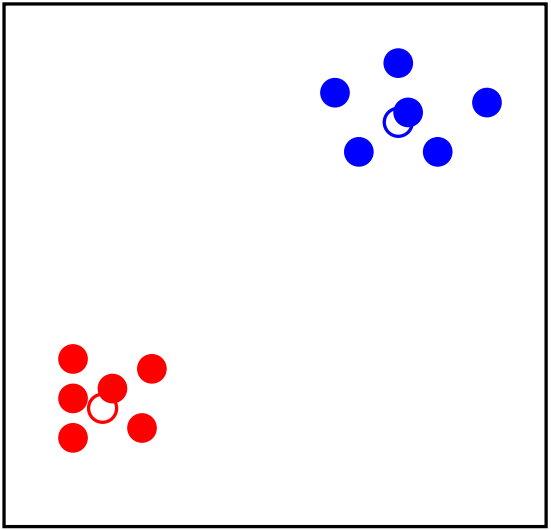
Lloyd's Algorithm Visualised

1) First, randomly assign every data point to one of the clusters.
2) Compute the mean position (in the data space) of each cluster
3) **For each data point, calculate distance to mean of each cluster**
4) **Re-assign points to their nearest cluster.**
5) Return to step 2 and repeat until the clusters are not changing.



Lloyd's Algorithm Visualised

1) First, randomly assign every data point to one of the clusters.
2) Compute the mean position (in the data space) of each cluster
3) For each data point, calculate distance to mean of each cluster
4) Re-assign points to their nearest cluster.
5) Return to step 2 and repeat until **the clusters are not changing.**



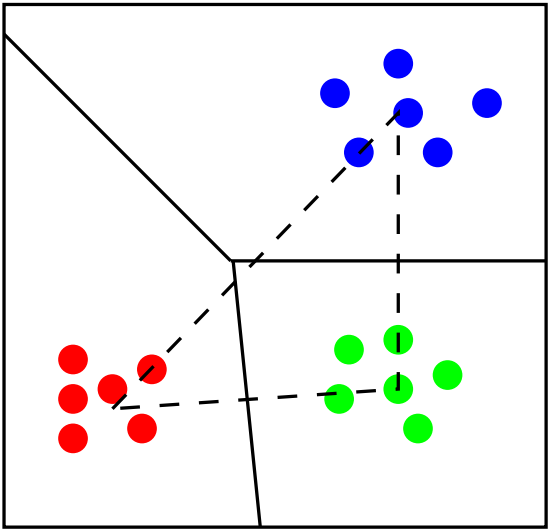
Notes about k-means

There are a few points to note:

- Initial random seeding $\rightarrow$ repeats needed
- Have to "guess" the number of clusters.
  - Too many, and you will split what should be a single cluster.
  - Too few, and you will combine clusters that should be separate, or splitting clusters and merge with others.
- This is a computationally "hard" algorithm (in the jargon, it is NP-hard).
  - The number of distance calculations increases rapidly with the number of clusters and the number of data points so it can be very costly for large datasets.
  - But not as hard as methods that compare all data points with each other

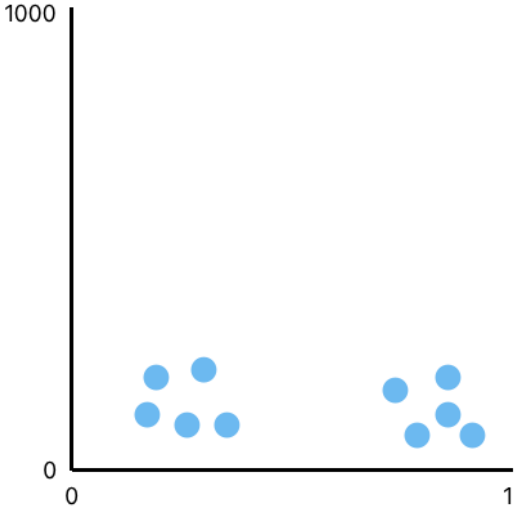
Notes about k-means

- Clusters are separated by linear boundaries only
- Regions define by these lines define the clusters



Notes about k-means

- Clusters are separated by linear boundaries only
- Regions define by these lines define the clusters
- Variables may need to be rescaled
- Common to scale to unit variance in each dimension



Summary

- Prototype methods are useful in situations where you have multiple phenomena present in a sample and want to distinguish between them
- Each data point $\rightarrow$ a distinct identifiable phenomenon
  - Cell or tissue type
  - Health vs diseased samples
  - Treatment vs no Treatment
- What if each datapoint is a mixture of phenomena *with their own signatures*?
  - Spectroscopic signal with multiple components
  - Multiple interacting genes
- We need a different approach that represents data points as mixtures of the Signatures
- And allows us to discover the component signatures too!

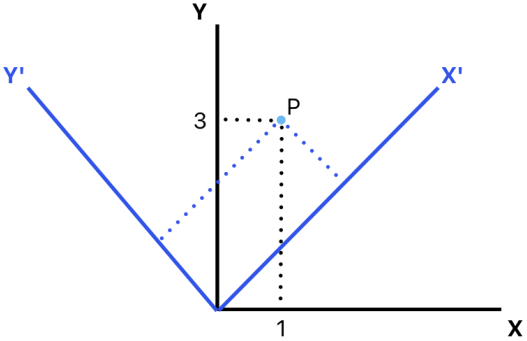
Decompositions for Signature Learning

- Take a set of n multivariate data points $\{\mathbf{X}^{(1)}, \ldots, \mathbf{X}^{(n)}\}$
- We can represent, or decompose, each data point as a sum of K components

$$\mathbf{X}^{(i)} = \sum_{k=1}^{K} A_{ik} \mathbf{\Phi}^{(k)} \quad \Longleftrightarrow \quad X_j^{(i)} = \sum_{k=1}^{K} A_{ik}$$

Or in matrix form, $\mathbf{X} = \mathbf{A}\mathbf{\Phi}$

Linear Decompositions



$$P = (1, 3) = \mathbf{X} + 3\mathbf{Y} = A_0\mathbf{X}' + A_1\mathbf{Y}'$$

Linear Decompositions

- We aim to find:
  1. The **components** Φ
  2. The **coefficients** A
- Infinitely many possibilities
- How do we choose them in a principled way?

- Independent Component Analysis (ICA)
- Random Projections
- Network Component Analysis
- Directional Component Analysis

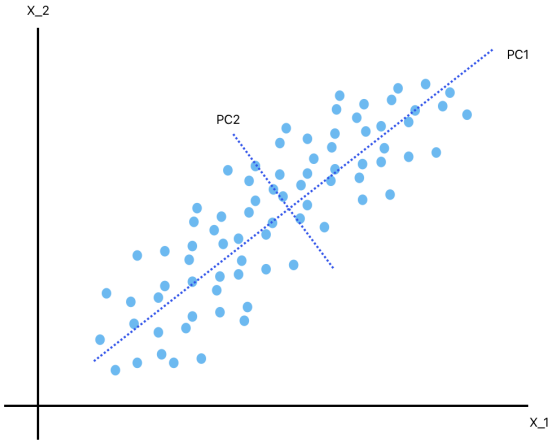
Linear Decompositions

- We aim to find:
  1. The **components** Φ
  2. The **coefficients** A
- Infinitely many possibilities
- How do we choose them in a principled way?

- Independent Component Analysis (ICA)
- Random Projections
- Network Component Analysis
- Directional Component Analysis
- **Principal Component Analysis (PCA)**
- **Non-negative Matrix Factrisation (NNMF)**

Principal Component Analysis

- PCA allows us to find a decomposition that aligns components to the shape of the data:
  - The first coordinate is aligned with the direction of greatest variation in the data.
  - The second coordinate is perpendicular to the first coordinate and aligned with the direction of greatest variation under that constraint,
  - Subsequent coordinates are perpendicular to all that have already been defined, aligned with the direction of next-greatest variation under that constraint.



Performing PCA

- Covariance matrix, $\mathbf{\Sigma}$ is central to PCA

$$\Sigma_{ij} = \frac{1}{N-1} \sum_{n=1}^{N} \left(x_i^{(n)} - \mu_i \right) \left(x_j^{(n)} - \mu_j \right)$$

- Easy to compute $\mathbf{\Sigma}$ if we organise our dataset in a sensible way:
  - Construct matrix $\mathbf{X}$ with components $X_{ij} = x_i^{(j)}$
  - Each *row* of $\mathbf{X}$ corresponds to a variable
  - Each *column* corresponds to a sample
  - Compute the mean of every row and tile the resulting column vector side-by-side N times $\rightarrow \bar{\mathbf{X}}$.
  - Then, we form the covariance matrix as

$$\mathbf{\Sigma} = \frac{1}{N-1} \left(\mathbf{X} - \bar{\mathbf{X}} \right) \left(\mathbf{X} - \bar{\mathbf{X}} \right)^{\mathrm{T}} = \frac{1}{N-1} \tilde{\mathbf{X}} \tilde{\mathbf{X}}^{\mathrm{T}} \tag{1}$$

Performing PCA

- We will now find coordinate vectors that align with the covariance structure
- Define matrix $\mathbf{T} = \begin{pmatrix} \mathbf{t}_1 & \mathbf{t}_2 & \dots \mathbf{t}_M \end{pmatrix}$
- Columns of $\mathbf{T}$ (the vectors $\mathbf{t}$) represent form a new coordinate system (basis) for the space.
- Normalised such that $\mathbf{T}^{\mathrm{T}}\mathbf{T} = \mathbf{I} \leftrightarrow \mathbf{t}_i \cdot \mathbf{t}_j = \delta_{ij}$.
- First we find the direction of greatest variance
- Find the vector $\mathbf{t}_1$ that maximises the variance of the projection of the data.

$$\mathbf{t}_1 = \arg\max_{\mathbf{t}} \mathrm{var}(\mathbf{t}^{\mathrm{T}}\mathbf{X})$$

- Now $\mathrm{var}(\mathbf{t}^{\mathrm{T}}\mathbf{X}) = \mathbf{t}^{\mathrm{T}}\tilde{\mathbf{X}}\tilde{\mathbf{X}}^{\mathrm{T}}\mathbf{t} = \mathbf{t}^{\mathrm{T}}\mathbf{\Sigma}\mathbf{t}$ ignoring the pre-factor $1/(N-1)$
- Find $\mathbf{t} = \mathbf{t}_1$ that maximises this subject to $\mathbf{t}^{\mathrm{T}}\mathbf{t} = 1$

Performing PCA

- Mathematical "trick": Lagrange multiplier - allows us to enforce constraints
- Subtract a "multiple of zero":

$$\mathbf{t}_1 = \arg\max_{\mathbf{t}} \left[\mathbf{t}^{\mathrm{T}}\boldsymbol{\Sigma}\mathbf{t} - \lambda_1(\mathbf{t}^{\mathrm{T}}\mathbf{t} - 1)\right] \tag{2}$$

- Differentiate this with respect to λ and set to zero to maximise

$$(\boldsymbol{\Sigma} - \lambda_1\mathbf{I})\,\mathbf{t}_1 = 0 \quad \rightarrow \quad \boldsymbol{\Sigma}\mathbf{t}_1 = \lambda_1\mathbf{t}_1 \tag{3}$$

- Solve "*eigenvalue*" problem for largest λ_1 and accompanying $\mathbf{t}_1$
- $\mathbf{t}_1$ is direction of greatest variation
- $\mathbf{t}_1^{\mathrm{T}}\boldsymbol{\Sigma}\mathbf{t}_1 = \lambda_1\mathbf{t}_1^{\mathrm{T}}\mathbf{t}_1 = \lambda_1$ is variance along that direction
- Next-greatest direction of variation is given by the second-largest eigenvalue/vector pair...

The principal components of a dataset are the eigenvectors of the covariance metric, ordered by eigenvalue.

- Once PCs have been found we transform the zero-mean data into the new coordinate system

$$\mathbf{X}' = \mathbf{T}^{\mathrm{T}}\tilde{\mathbf{X}} \tag{4}$$

- $\sqrt{(}\lambda_i)\mathbf{t}_i$ are called the **loadings**.
- Columns of $\mathbf{X}'$ are the **scores**: data projected onto the PC axes.
- Note that $\mathbf{\Sigma} = \mathbf{TST}^{\mathrm{T}}$ where $\mathbf{S} = \mathrm{diag}(\lambda_1, \lambda_2, \ldots, \lambda_n)$.

Reducing dimensionality with PCA

- Each PC is a linear combination of the original variables
- Because PC's are aligned with data, do not always need all of them
- We can remove non-varying dimensions.
- Useful to consider the "total variance" $V_{\mathrm{T}} = \sum_i \lambda_i$.
- Compute proportion of total variance along each PC by computing $\lambda_i / V_{\mathrm{T}}$.
- Determine how many PCs describe the majority of the variance.
- Decide which ones to remove
- Set components of data in those dimensions to zero or equivalently, remove the corresponding columns from $\mathbf{T}$.

Data Normalisation and Variable Scaling

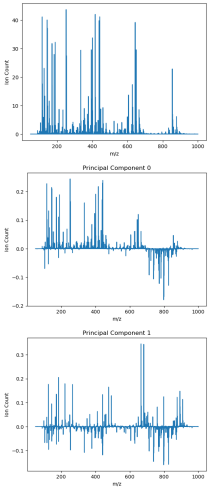
- In some multivariate datasets, variables are of a different type or on a different scale
- Useful to consider mapping all variables onto a common scale to avoid bias
- Most common: rescale all variables to have unit variance.
- Take care when using PCA in such a situation.

PCA and signatures

- The components (loadings) of PCA can be regarded as signatures
- Each component represents some fundamental building block of the data

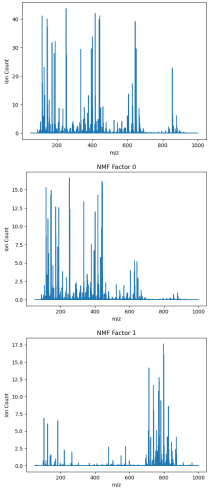
A Problem with PCA

- PCA components are linear combinations of the original variables
- The loadings of those can be positive or negative
- In many situations, this is unphysical
- Many variable types cannot take negative values
  - Chemical concentrations
  - Numbers of specimens
  - Volumes, masses, etc
- PCA loadings not always physically interpretable



Non-Negative Matrix Factorisation

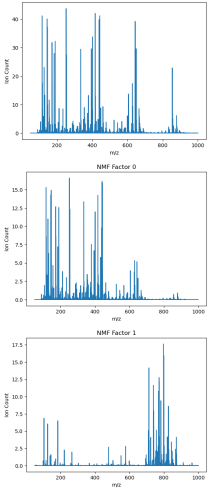
- In non-negative matrix factorisation we aim to find a more interpretable decomposition
- Components are positive linear combinations of variables
- Samples are positive linear combinations of components



Non-Negative Matrix Factorisation

- NNMF is not as clean as PCA
- No analytic solutions
- Requires approximation and numerics
- NNMF is typically an **approximate** decomposition

$$X_j^{(i)} = \sum_{k=1}^{K} A_{ik} \Phi_j^{(k)}$$

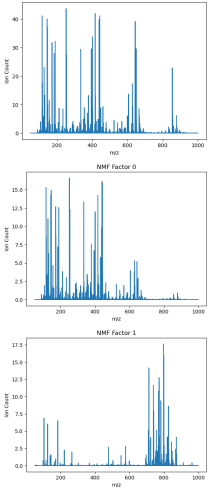


Non-Negative Matrix Factorisation

- NNMF is not as clean as PCA
- No analytic solutions
- Requires approximation and numerics
- NNMF is typically an **approximate** decomposition

$$X_j^{(i)} \approx \sum_{k=1}^{K} A_{ik} \Phi_j^{(k)}$$

with A_{ik} and $\Phi_j^{(k)}$ all strictly ≥ 0.



Computing analysis NNMF

- The general problem statement is $\mathbf{X} \approx \mathbf{A}\mathbf{\Phi}$ with $\mathbf{X}$ known
- The elements of $\mathbf{X}$, $\mathbf{A}$, $\mathbf{\Phi}$ are all ≥ 0
- Need to solve $\arg\min_{\mathbf{A},\mathbf{\Phi}} \|\mathbf{X} - \mathbf{A}\mathbf{\Phi}\|^2$ under positivity constraints
- Can be solved via linear programming, convex optimisation, and other methods

Non-linear Representations

- PCA and NNMF are purely linear transformations of the data
- Do not change the "shape" of a cloud of datapoints
- Not suited for finding finer structure in a dataset
- Popular alternatives for this are:
  - Self-organising maps
  - Curvilinear component analysis
  - Latent variable models
  - t-distributed stochastic neighbour embedding
  - Uniform Manifold Approaximation and Projection (UMAP)
- Often use linear methods as pre-processing to eliminate globally redundant dimensions
- Then use a non-linear method or clustering to sort out the subleties.